

Agile Methodologies: Scrum & XP

Not from your notes — drafted fresh. The Requirements Engineering & Agile topic covered agile *principles* generically (the Agile Alliance's 12 principles, human factors) but never named a specific methodology. Scrum and Extreme Programming (XP) are the two agile frameworks most commonly asked about **by name** in interviews, so this draft covers them concretely.

1. Scrum vs. XP — the One-Sentence Distinction

Scrum is a project-management framework — it defines *roles*, *ceremonies*, and *artifacts* that structure *when* and *by whom* work gets planned, done, and reviewed. **XP (Extreme Programming) is a set of engineering practices** — it defines concrete *technical habits* for *how* code actually gets written, tested, and integrated.

They are **complementary, not competing** — a huge number of real-world teams run "Scrum as the process wrapper" (sprints, standups, a product owner) while also adopting XP's engineering practices (pair programming, TDD, continuous integration) *inside* that wrapper. Scrum tells you the team meets every morning; XP tells you what good code discipline looks like once they sit down to write it.

2. Scrum

The Three Roles

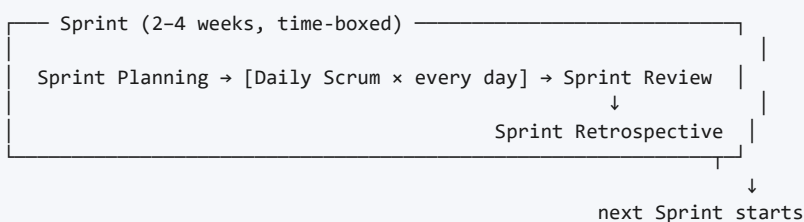
Role	Responsibility	Is it a manager?
Product Owner	Owns and prioritizes the Product Backlog; represents the customer/business voice; decides <i>what</i> gets built and in what order	No — owns priority, not the team's time or methods
Scrum Master	Facilitates the process, removes impediments/blockers, shields the team from external disruption, coaches Scrum practice	No — a servant-leader/facilitator role, not a boss with authority over the team
Development Team	Self-organizing, cross-functional; decides <i>how</i> to build the increment	N/A — collectively self-managing

Why isn't the Scrum Master a manager? Because Scrum's core bet is that a self-organizing team produces better designs and makes better day-to-day decisions than one directed top-down (this is literally Agile principle #11 from the Requirements Engineering & Agile topic) — a Scrum Master who started assigning tasks or dictating technical decisions would undermine the entire premise. Their job is to clear *obstacles in the team's way* (a blocked dependency, an unresponsive stakeholder, a broken build pipeline) — not to decide what the team works on or how.

The Three Artifacts

- **Product Backlog** — a single, prioritized list of everything that could possibly be built, owned and ordered by the Product Owner, continuously refined as understanding improves (this is Requirements Engineering's Elicitation and Elaboration tasks, running continuously rather than once).
- **Sprint Backlog** — the subset of Product Backlog items selected for the current sprint, plus the team's plan for delivering them.
- **Increment** — the sum of everything completed during the sprint (and all prior sprints), which must be in a **potentially shippable** state — "done" means genuinely done, not "done except for testing" or "done except for polish."

The Events



- **Sprint** — a fixed-length (typically 2–4 weeks) time-box during which a "done," usable increment is produced. The length doesn't change sprint to sprint — consistency here is what makes velocity/burndown tracking meaningful.
- **Sprint Planning** — the team selects which Product Backlog items to pull into this sprint and plans how they'll get done.
- **Daily Scrum (Standup)** — a short (~15 min), same-time-every-day sync, traditionally structured around three questions: what did I do yesterday, what will I do today, what's blocking me. It exists to **synchronize the team**, not to report status upward to a manager — if it turns into a status report *to* the Scrum Master or Product Owner, it's being run wrong.
- **Sprint Review** — the team demos the increment to stakeholders and gathers feedback, which feeds back into the Product Backlog. This is about the **product**.
- **Sprint Retrospective** — the team reflects on *how* they worked this sprint (the process, not the product) and identifies concrete improvements for next sprint. This is about the **process**.

Review vs. Retrospective, one line: Sprint Review asks "is this the right *product*?" (stakeholders look at the increment); Sprint Retrospective asks "are we working the right *way*?" (the team looks at itself). Mixing them up is a very common beginner mistake — a retrospective that turns into a product demo, or a review that turns into the team debating its own workflow, wastes both meetings' actual purpose.

Supporting Concepts

- **Story Points** — a *relative* unit of estimation (not hours) reflecting complexity, effort, and uncertainty together — the point is comparing items to each other ("this is about as complex as that other 5-point story we did last sprint"), not predicting exact calendar time, because relative comparison is something teams are demonstrably better at than absolute time estimation.
- **Burndown Chart** — plots remaining work (in story points) against time remaining in the sprint; an ideal burndown is a straight diagonal line to zero, and a real one that's flat for days then drops suddenly is a visible signal something's off (blocked work, or work being marked "done" in a big batch at the end).
- **Definition of Done** — a team-wide, explicit, shared checklist of what "complete" actually means for any backlog item (e.g., code written, reviewed, tested, merged, documented) — without one, "done" silently means something different to each team member, which is exactly the kind of ambiguity Requirements Validation warns against, just applied to internal team communication instead of stakeholder requirements.

3. Extreme Programming (XP)

XP's practices cluster around progressively **tightening feedback loops** — the shorter the gap between writing something and finding out whether it's right, the cheaper mistakes are to fix (directly connecting to the cost-of-change discussion from the Requirements Engineering & Agile topic).

Feedback loop length:	Practice:
Seconds	Pair Programming (immediate second opinion)
Minutes	TDD, Continuous Integration (immediate test feedback)
Days	Small Releases, On-Site Customer (fast real-world feedback)
Weeks	Planning Game / iteration cycle

Planning Practices

- **Planning Game** — customer and developers collaboratively decide the scope and priority of the next release: the customer describes desired features and their business value, developers estimate technical effort and risk, and scope is negotiated jointly — essentially a formalized version of the Negotiation task from Requirements Engineering.
- **Small Releases** — release small, frequent, genuinely valuable increments — even smaller and more frequent than a typical Scrum sprint, in XP's original formulation.

- **On-Site Customer** — a real customer representative is embedded with the team, physically available, so questions about requirements get answered in minutes rather than days.

Design Practices

- **Simple Design** — "do the simplest thing that could possibly work" — closely related to **YAGNI** ("You Aren't Gonna Need It"): don't build flexibility or abstraction for a hypothetical future requirement that doesn't exist yet (this is the same anti-over-engineering instinct behind avoiding premature abstraction generally).
- **Refactoring** — continuously improving the code's internal structure without changing its behavior (see the Design Concepts topic) — XP treats this as a *constant*, ongoing activity, not an occasional cleanup pass.

Coding Practices

- **Pair Programming** — two developers share one workstation: a **driver** actively types, while a **navigator** reviews each line in real time, thinks ahead about edge cases and design implications, and catches mistakes before they're even committed. Roles swap frequently.
- **Collective Code Ownership** — any developer can (and is expected to) modify any part of the codebase when needed, rather than code being siloed to whoever originally wrote it.
- **Coding Standards** — shared, team-wide conventions so code reads uniformly regardless of who wrote it — this is a *prerequisite* for collective code ownership actually working smoothly (see Q6 below).
- **Continuous Integration (CI)** — integrate and test the whole system frequently — multiple times a day, not once a week — specifically so integration conflicts and defects surface within hours of being introduced, not weeks later when they're much harder to trace back to their cause.

Testing Practice

- **Test-Driven Development (TDD)** — write the failing test before the code (see the Software Testing topic for the full Red-Green-Refactor cycle). TDD is arguably XP's signature practice.

Sustainable Process

- **Sustainable Pace ("40-hour week")** — explicitly rejects chronic overtime as a substitute for good planning; directly implements Agile principle #8 ("sustainable development, at a constant pace") from the Requirements Engineering & Agile topic.
-

4. Quick Comparison

	Scrum	XP
Primary focus	Process / project management — who does what, when	Engineering discipline — how the code itself gets written
Defines roles?	Yes — Product Owner, Scrum Master, Dev Team	No prescribed roles beyond "the customer" and "the team"
Core mechanism	Ceremonies (sprint planning, standup, review, retro) + artifacts (backlogs, increment)	Concrete technical practices (pairing, TDD, CI, refactoring, simple design)
Typical iteration	Sprint, usually 2–4 weeks, fixed length	Originally shorter iterations, very frequent small releases
Can they combine?	—	Very commonly run together: Scrum's ceremonies as the outer structure, XP's engineering practices as what actually happens inside each sprint

Interview Questions With Answers

Q1. A candidate says "Scrum and XP are two competing agile frameworks, and a team has to choose one." What's wrong with this framing?

Answer: This framing treats Scrum and XP as if they answer the same question, when they actually answer two different questions: Scrum answers "how is our work organized and reviewed over time" (roles, sprints, ceremonies, backlogs), while XP answers "what technical habits produce good code" (pair programming, TDD, continuous integration, refactoring). Because they operate at different layers — one is a project-management/process framework, the other is a set of engineering practices — they aren't mutually exclusive alternatives at all; in practice, many teams run Scrum's sprint/ceremony structure as their outer process while simultaneously adopting XP's engineering practices as how they actually write code within each sprint, getting the benefits of both rather than being forced to pick one.

Q2. Why does the Daily Scrum specifically avoid being a "status report to the Scrum Master or Product Owner," and what would it look like if that rule were broken?

Answer: The Daily Scrum exists to synchronize the *development team* with each other — surfacing blockers early and coordinating who's working on what — which requires the team to be talking to each other, not reporting upward to an authority figure who then makes decisions on their behalf. If it turned into a status report to the Scrum Master or Product Owner, it would look like each team member addressing their update *to* that one person, waiting for their reaction or direction, rather than team

members reacting to *each other's* updates — subtly reintroducing a top-down management dynamic into a role (Scrum Master) that's explicitly defined as a facilitator rather than a boss, and undermining the self-organizing team principle the entire Scrum framework is built around.

Q3. Why does Scrum use relative story points rather than having the team estimate tasks directly in hours?

Answer: Estimating absolute time (hours or days) for a not-yet-built piece of software requires accurately predicting unknowns — how long debugging will take, how many edge cases will surface, how well a particular approach will actually work — that are inherently hard to predict precisely, and human estimators are demonstrably bad at this kind of absolute prediction, especially for unfamiliar work. Story points sidestep this by asking a comparative question instead — "is this roughly as complex as that other story we already agreed was a 5?" — which people are much more reliable at judging, since it doesn't require predicting an unknown quantity from scratch, only comparing relative difficulty against a known reference point. Over several sprints, a team's actual "velocity" (story points completed per sprint) becomes a track record that lets you forecast future sprints statistically, without ever having needed accurate absolute time estimates for any individual story.

Q4. Pair Programming puts two developers on one task at the same time — doesn't that just double the cost of writing code? How would you defend it in an interview?

Answer: On a naive per-line-of-code basis, yes, two people working simultaneously on one task costs more developer-hours than one person working alone — but that framing ignores what pairing actually buys: the navigator catches design flaws, logic errors, and edge cases *in real time*, before they're ever committed, rather than those issues surfacing later in code review, QA, or (worse) production, each of which is progressively more expensive to fix per the cost-of-change principle. Pairing also spreads knowledge across the team continuously (supporting Collective Code Ownership — see Q6) rather than concentrating it in whoever happens to write a given piece of code, which reduces the "bus factor" risk of one person being the only one who understands a critical piece of the system. The honest defense isn't "pairing is free" — it's that the *upfront* cost of two people is often smaller than the *downstream* cost (defects escaping to later stages, knowledge silos, bottlenecked reviews) that pairing prevents.

Q5. Explain how Collective Code Ownership depends on Coding Standards and Continuous Integration to actually work, rather than being an independent practice.

Answer: Collective Code Ownership means any developer can modify any part of the codebase — but if every part of the codebase were written in a different developer's personal style, with inconsistent conventions, a developer touching unfamiliar code would face a real cognitive barrier just parsing what they're looking at before they could safely change it, which would undermine the whole point of the practice. Coding Standards remove that barrier by making the entire codebase read uniformly regardless of author, so "unfamiliar code" is still recognizably *the same kind of code* the developer already knows how to work with. Continuous Integration provides the second half of the safety net: because any developer might now be modifying code they didn't originally write, there needs to be a fast, reliable way to catch it immediately if a change breaks something elsewhere in the system — which

is exactly what frequent, automated integration and testing provides. Without both of these supporting practices, collective code ownership in isolation would likely just produce a codebase where developers are afraid to touch anything they didn't personally write, defeating its purpose.

Q6. What specifically distinguishes a Sprint Review from a Sprint Retrospective, and why does confusing the two waste both meetings?

Answer: A Sprint Review is about the **product** — the team demonstrates the actual working increment to stakeholders, who give feedback that feeds directly back into the Product Backlog for future prioritization. A Sprint Retrospective is about the **process** — the team, without external stakeholders present, reflects on how they worked together during the sprint (communication, tooling, blockers, workflow) and commits to concrete process improvements for the next sprint. Confusing them wastes both: if a Review turns into the team debating their own internal workflow, stakeholders — who came specifically to see and react to the product — get a meeting that isn't about what they came for; if a Retrospective turns into re-demoing the product or re-litigating feature decisions, the team loses its one dedicated opportunity to honestly examine and improve *how* they work, which is a fundamentally different (and, without a dedicated slot, easily neglected) kind of reflection.

Q7. A Product Owner starts telling the Development Team exactly how to implement a feature — which classes to create, which algorithm to use. What Scrum principle does this violate, and why does it matter beyond just "role confusion"?

Answer: This violates the separation between deciding **what** gets built (the Product Owner's role, via backlog prioritization) and deciding **how** it gets built (the self-organizing Development Team's role) — the Product Owner dictating implementation details is overreaching into technical decisions the framework specifically reserves for the team closest to the actual code. This matters beyond simple role confusion because it undermines the core empirical bet behind self-organizing teams (Agile principle #11): that the people actually doing the technical work are best positioned to make good technical decisions, and that ownership over *how* to solve a problem increases both the quality of the solution and the team's genuine investment in it. A Product Owner who dictates implementation details is effectively reintroducing top-down command-and-control into exactly the part of the process Scrum is designed to keep bottom-up — and doing so both produces worse technical decisions on average (since the PO isn't necessarily the person best equipped to make them) and erodes the team's sense of ownership over their own work.

Q8. Scenario: A team using Scrum has good sprint ceremonies (planning, standups, reviews, retros all happen reliably) but is still shipping frequent regressions and struggling with slow, painful merges every sprint. What's the most likely underlying gap, and which framework would you look to for a fix?

Answer: Reliable ceremonies with poor code-quality outcomes strongly suggests the gap isn't in the team's *process* structure (which Scrum governs and which is apparently working) but in their *engineering practices* — precisely the layer Scrum deliberately doesn't prescribe anything about. Frequent regressions point toward inadequate automated testing and insufficient regression coverage;

painful, infrequent merges point toward long-lived branches and infrequent integration rather than the small, frequent integration XP recommends. The fix here is to look to **XP**, not to add more Scrum ceremonies (which wouldn't touch either problem) — specifically adopting Continuous Integration (merging and testing frequently, in small increments, rather than accumulating large, conflict-prone changes) and Test-Driven Development (building a genuine regression safety net as a byproduct of how code gets written, rather than testing being an afterthought). This scenario is a good illustration of exactly why "Scrum vs. XP" is the wrong framing (per Q1) — a team can be doing Scrum's process correctly and still need XP's engineering practices to solve a problem Scrum was never designed to address in the first place.